Everyday

Ecommerce with an AI shopping assistant

Project overview and low-level design



11 October 2026

Shopping journey

Visitors browse; authenticated shoppers save products and create unpaid orders

Browse
Search and sort

→

Sign in
Register or login

→

Build a bag
Buttons or chatbot

→

Review
Items and shipping

→

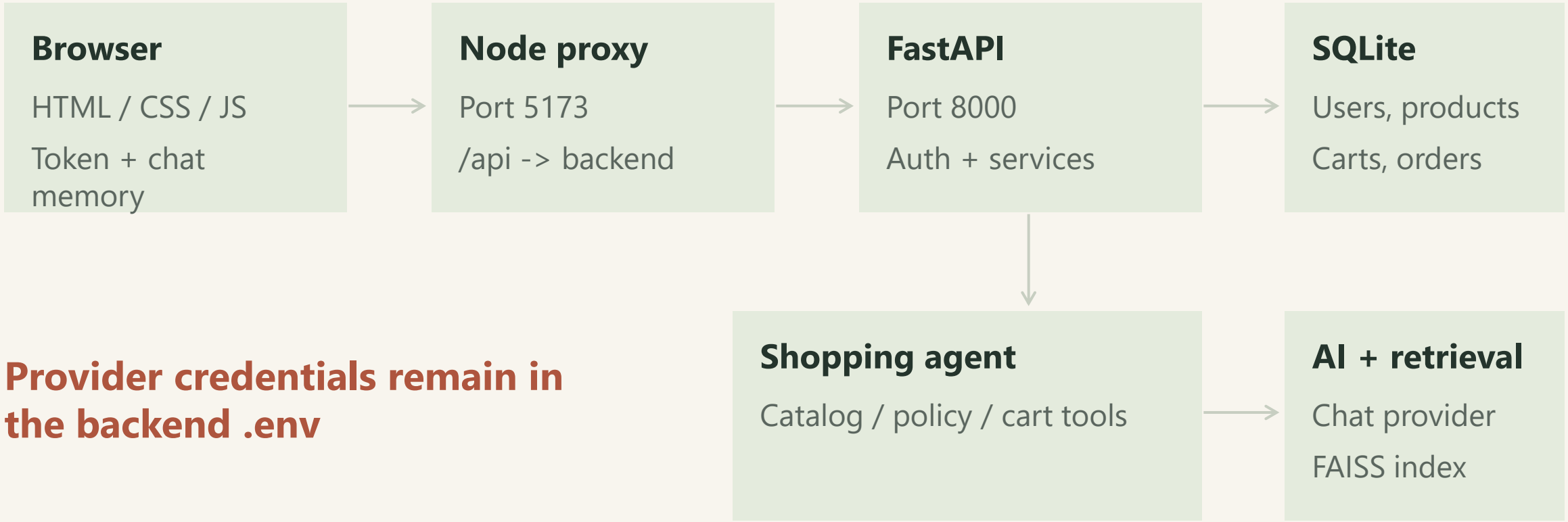
Save an order
Pending payment

Ask Everyday

Answers catalog questions, retrieves policy passages, and adds requested products to your own bag.

System architecture

The browser uses a same-origin Node proxy to reach FastAPI



Provider credentials remain in the backend .env

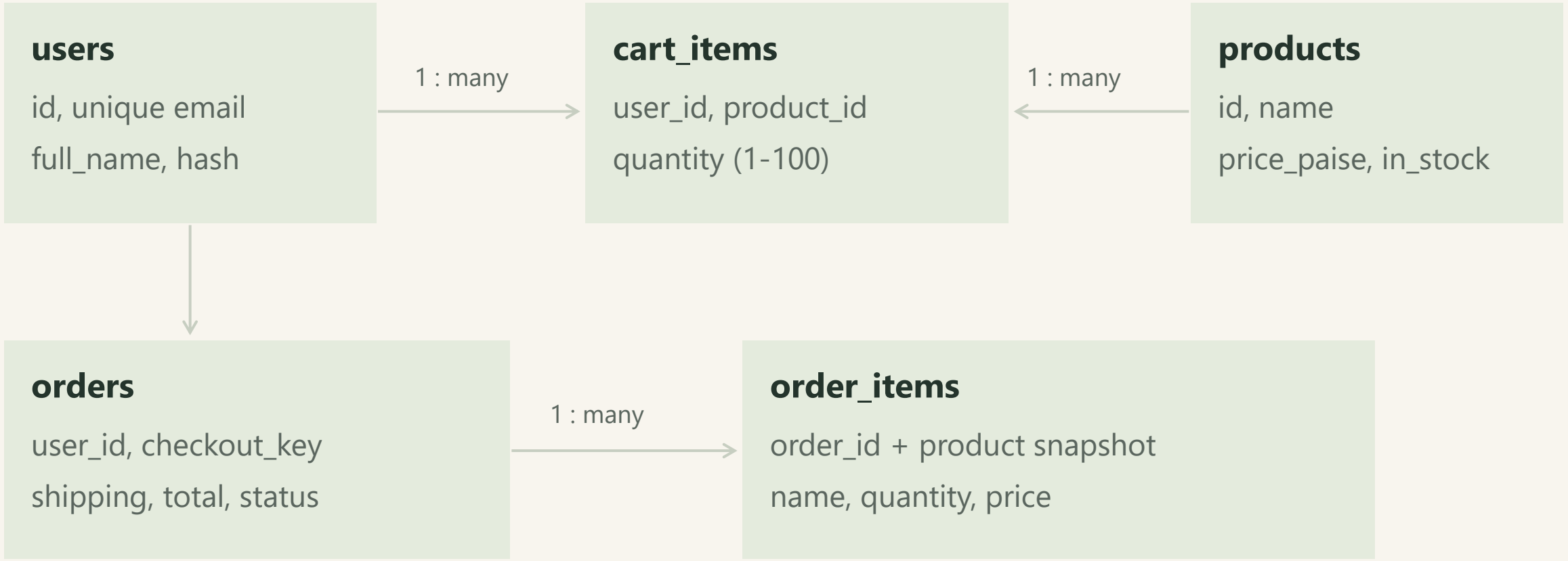
Module responsibilities

HTTP contracts, operations, persistence, and model tools have separate files

Layer	Responsibility	Source location
Frontend	State, forms, dialogs, rendering	frontend/public/app.js
Routers + schemas	HTTP status and input validation	routers/ + schemas/
Services	Cart, checkout, auth operations	services/
Models + database	Tables and SQLModel sessions	models/ + database.py
AI	Agent, tools, model and RAG	ai/
Core	Configuration and bearer auth	core/

Database relationships

Five tables store account data, the current bag, and immutable order snapshots



Authentication and ownership

Every cart, order, and chat operation resolves the user on the server

01 **Verify password**

Argon2 hash; consistent error for invalid login

02 **Issue access token**

HS256 JWT with sub, iat, exp; 30-minute expiry

03 **Validate each request**

Signature, expiration, required claims, existing user

04 **Bind ownership**

Use CurrentUser.id; never trust a supplied user ID

Automatic sample catalog

Startup adds 12 missing demo products without overwriting existing records

Sample group	Example products
Stationery	Notebook, pen set
Home + drinkware	Mug, water bottle, lamp, candle
Clothing + bags	T-shirt, tote bag
Electronics	Mouse, cable, Bluetooth speaker
Fitness	Yoga mat

Name matching avoids restart duplicates. Edited prices and stock remain unchanged.

Cart behavior

The HTTP endpoints and assistant reuse the same cart service

Shopper actions

Add a quantity to an existing line

Replace quantity or remove an item

See current prices and subtotal

Server checks

Authenticated cart owner

Existing and available product

Maximum 100 units per product

Deleted products remain removable

Agent adds a product to the bag

Example: "Add 2 ceramic coffee mugs to my cart"

1. Authenticate

Resolve bearer token

2. Find product

list_products -> actual ID

3. Call cart tool

Product ID + quantity only

Flow continues from step 3 to step 4

4. Save to SQLite

User ID bound by server

5. Reply

Confirm tool success

6. Refresh bag

Update count and contents

Conversation memory and policy retrieval

Conversation context and store knowledge follow different paths

Chat context

Browser memory: chatHistory[]

Last 12 successful messages

Sent with each new message

Cleared on refresh or sign-out

No persistent agent checkpointer

Policy knowledge

Policy PDF -> chunks -> FAISS

1,000 characters / 200 overlap

Retrieve four similar passages

Return source + page numbers

Manual ingest; live products use SQL

Checkout and order replay

One transaction saves snapshots and clears the bag

01 **Review items and shipping**

Client sends quantities, prices, total, and a UUID

02 **Lock and compare**

SQLite BEGIN IMMEDIATE; verify current catalog/cart

03 **Save atomically**

Insert pending_payment order + snapshots; clear cart

04 **Replay safely**

Same UUID + same payload returns the existing order

Failure handling

The UI explains recoverable errors without exposing provider secrets

HTTP	Condition	Shopper action
401	Missing, invalid or expired login	Sign in again
404	Missing / unowned cart or order	Refresh the view
409	Unavailable product or changed cart	Review the bag
422	Invalid field or message limits	Correct the input
503	Assistant provider failure / timeout	Check bag before retry
502	Backend proxy unavailable	Retry after connection recovers

Demo and next steps

The current app is a local ecommerce demonstration

Demo sequence

Start backend and frontend

Browse the mixed sample catalog

Sign in and ask to add a mug

Review the bag; save an unpaid order

Open My orders

Future work

Administrator access for catalog writes

Product photos and descriptions

Persistent chat and action retry IDs

Verified payments and stock quantities